

Внутреннее файловое хранилище MK61s mini

Версия документа: 06.09.2026

MK61s mini хранит программы, тексты, состояния, шрифты, изображения, ROM виртуальных консолей и каталоги во внешней SPI NOR flash. Постоянным источником истины является собственная файловая система C5 (program_store). USB Mass Storage показывает её как динамически сформированный FAT12-диск (virtual_fat). Готового FAT-образа во flash нет.

C5 намеренно не читает прежние форматы тома C1--C4. Эти исторические имена форматов flash не следует путать с двухбайтовой сигнатурой типа C1, назначенной ROM CHIP-8. При первом запуске этой версии старое хранилище считается неразмеченным, реальная ёмкость flash измеряется, после чего создаётся новая C5. Миграции и восстановления старых файлов нет.

Host-тест подставляет реальные сигнатуры S1/C1--C4 всех прежних форматов, проверяет однократный переход в C5 и повторную загрузку без новых erase/program. При создании C5 немедленно стирается только публикуемый catalog bank, сектор настроек и две locator-копии. Второй COW bank, data log и USB staging защищены format epoch и очищаются лениво перед первым использованием; это устраняет более ста лишних 4-КиБ erase на первом запуске W25Q128.

Эталонная реализация находится в:

- code/spi_nor_flash.cpp -- JEDEC/SFDP и низкоуровневый SPI NOR;
- code/flash_capacity_probe.hpp -- проверка физической границы;
- code/storage_geometry.cpp -- расчёт C5 и FAT12;
- code/program_store.cpp -- каталог, WAL, данные, GC и USB staging;
- code/storage_path.cpp -- единый разбор абсолютных и относительных путей;
- code/virtual_fat.cpp -- FAT12-проекция и импорт изменений хоста.

Основные свойства

Параметр	C5
Физический erase-сектор	4096 байт
Логический сектор USB	512 байт
Поддерживаемая ёмкость SPI NOR	128 КиБ--128 МиБ
Максимальная длина программы/текста	1536 байт; TinyBASIC — 3584 байта
Максимальная длина FMK	1536 байт на F401; 8192 байта на F411
Максимальная длина изображения WBMP	1600 байт
Максимальная длина ROM CHIP-8	3584 байта
Максимальная длина APP-контейнера	20 544 байта
Максимальная длина basename	31 байт UTF-8
Максимальная глубина каталогов	32
Копий FAT	2
Максимум кластеров данных FAT12	4084
Зарезервировано под настройки	1 erase-сектор
Доступно журналу настроек	4080 байт
Одновременно staged USB-блоков	384 по 512 байт на flash от 2 МиБ; 96 на 512 КиБ

Квота объектов больше не фиксирована. Она вычисляется из измеренной ёмкости, физического worst case и границы FAT12. На двух целевых микросхемах:

Физическая flash	Узлов C5	Размер FAT-кластера	Секторов FAT12
512 КиБ	192	2 КиБ	832
16 МиБ, W25Q128	4084	4 КиБ	32729

Узел -- файл, каталог либо скрытый extent большого каталога/многокластерного APP или ROM. Поэтому 4084 -- предельная квота всего тома, а не обещание 4084 файлов непосредственно в корне. Каталог сам занимает один узел. Дополнительный extent расходуется, когда его записи не помещаются в один FAT-кластер.

Лимит программ и текстов равен 1536 байтам, кроме исходников TinyBASIC: они могут занимать 3584 байта и всё ещё помещаются с метаданными в один 4-КиБ сектор C5. F401 сохраняет предел 1536 байт для FMK. На F411 шрифт может занимать до 8192 байт: это позволяет загружать полноценный русский UI-растр прямо в уже существующую общую арену BULK, без второго буфера. Если FMK больше 1600 байт, C5 хранит его тем же потоковым многосекторным способом, что крупные APP и ROM. Только изображения WBMP получают 1600 байт: дополнительные 64 байта позволяют хранить полный кадр 192x64 вместе с заголовком и по-прежнему помещаются в минимальный 2-КиБ виртуальный кластер. Сырой ROM CHIP-8 может занимать до 3584 байт - остаток 4-КиБ адресного пространства после точки загрузки 0x200. APP выделен в отдельный потоковый тип: 64-байтовый заголовок плюс до 20 КиБ сжатого или несжатого образа. Фиксированного числа APP или ROM нет; предел задают общие узлы, блоки и каталоги.

Определение физической ёмкости

Быстрый путь при обычной загрузке

Драйвер читает JEDEC ID командой 0x9F, затем заголовок и Basic Flash Parameter Table SFDP командой 0x5A. Значения сохраняются как заявленная геометрия и помогают выбрать протокол команд, но не ограничивают поиск: они не считаются доказательством физической ёмкости и могут быть ошибочными в обе стороны.

Если обе locator-копии уже относятся к текущему каталогу v6, загрузка является read-only:

- 1 Проверяются сигнатура, версия, committed-state и CRC32 locator.
- 2 Геометрия полностью пересчитывается и сравнивается с сохранёнными полями.
- 3 Сравниваются JEDEC ID и заявленная JEDEC/SFDP-граница.
- 4 На измеренном дальнем конце читается guard сектора настроек.
- 5 Драйвер переключается на сохранённую измеренную ёмкость.

Ни erase, ни тестовая запись на штатной загрузке не выполняются. Это сохраняет ресурс flash. Завышенная маркировка, однажды измеренная как меньшая физическая ёмкость, также не вызывает повторное форматирование при каждом включении. Исключение -- однократная COW-миграция каталога v5 в v6 или завершение прерванного обновления второй locator-копии; пользовательские данные при этом не переписываются.

Деструктивный пробник неразмеченной flash

Пробник запускается только тогда, когда нет валидного C5 locator. Содержимое такой микросхемы всё равно будет заменено новой C5, поэтому тестовые секторы не копируются и не восстанавливаются.

Стандартные ёмкости SPI NOR являются степенями двойки. Независимо от заявленной ёмкости по показателям степени от 17 до 27 выполняется настоящий двоичный поиск максимальной рабочей границы:

128 КиБ, 256 КиБ, 512 КиБ, ... 64 МиБ, 128 МиБ

Для проверки candidate используются два сектора:

```
lower = candidate / 2 - 4096
upper = candidate - 4096
```

Оба сектора стираются, затем в них записываются разные комплементарные 32-байтные метки. После обеих записей обе метки читаются и сравниваются. Если старшие адресные биты отсутствуют и upper зеркалит lower, вторая запись потребует невозможного для NOR перехода 0 -> 1 либо финальное

сравнение увидит alias. Ошибка команды, verify или таймаут также означает нерабочую границу.

Для диапазона 128 КиБ--128 МиБ требуется не более четырёх проверок candidate, то есть не более восьми основных erase-операций. При проверке границ выше 16 МиБ дополнительно защищаются два низких shadow-сектора. Это обнаруживает трёхбайтную микросхему, которая проигнорировала EN4B и приняла четвёртый байт адреса за данные. Таких защитных стираний не более шести за весь поиск; общий худший случай -- 14, а для микросхем до 16 МиБ их нет.

Стирания и page-program одной SPI NOR нельзя выполнять параллельно: кристалл выставляет BUSY. Реализация не делает лишнего промежуточного чтения -- обе метки программируются последовательно, а затем проверяются одним этапом чтения.

Поиск всегда включает 128 МиБ, поэтому заниженные JEDEC/SFDP не оставляют неиспользованную физическую ёмкость. Если не проходит даже 128 КиБ, C5 не создаётся.

Адресация больше 16 МиБ

Для микросхем с четырёхбайтным адресом драйвер разбирает optional SFDP 4-Byte Address Instruction Table (4BAIT). Если таблица одновременно объявляет обычные read, page-program и выбранный 4-КиБ erase type, используются stateless-команды 13h, 12h и указанный таблицей erase opcode. В остальных случаях драйвер разбирает BFPT DWORD 16 и использует B7h/E9h, в том числе вариант с обязательным WREN. Неизвестный способ переключения не угадывается: граница выше 16 МиБ в таком случае не принимается.

SFDP также задаёт размер program-page. Драйвер никогда не отправляет за одну операцию больше 256 байт и уменьшает порцию для микросхем с меньшей страницей, исключая page-wgap.

Горячий путь SPI использует буферные transfer-операции STM32 HAL: сектор читается одним вызовом, page-program -- одним вызовом на физическую страницу, а verify 512 байт -- одной READ-командой и одной порцией сравнения. Побайтового входа в HAL для каждого из 512 байт нет. Постоянная RAM при этом не расходуется; для verify используется ограниченный 512-байтный stack buffer.

Что именно гарантирует быстрый тест

Разреженный пробник доказывает адресуемый диапазон, отсутствие зеркалирования на проверяемых границах и работоспособность последнего сектора, отданного настройкам. Он намеренно не является полным тестом здоровья каждого сектора: такой тест потребовал бы стереть и записать всю микросхему и занял бы минуты.

Все реальные записи C5, включая locator, каталоги, WAL, данные и настройки, проверяются чтением. Поэтому отдельный изношенный сектор приводит к явному отказу операции, а не к подтверждению повреждённых данных; неиспользованные data-секторы проверяются лениво при первом обращении.

Единицы getCapacity()

getCapacity() возвращает байты. Старый диагностический код ошибочно печатал после числа суффикс К:

```
16777216 = 16 МиБ, W25Q128, а не 16777216 КиБ
524288   = 512 КиБ, а не 524288 КиБ
```

Новый вывод явно пишет bytes и отдельно показывает заявленную верхнюю и измеренную C5-ёмкость.

Динамическая карта SPI NOR

Все числа ниже -- номера 4096-байтных erase-секторов. Ни один сектор после измеренной границы не используется.

```
0          locator A
1          locator B
2 ..       catalog bank A
           catalog bank B
           packed data log
... settings-stage  USB staging log
capacity/4096 - 1   settings journal + C5 guard
```

Один catalog bank состоит из:

```
1 sector header + ceil(max_nodes * 20 / 4096) table sectors + 2 WAL sectors
```

Размер staging динамический: на flash от 2 МиБ это 65 секторов (64 рабочих и один резерв атомарной компактизации), на 512 КиБ -- 17, а ниже -- 4--8. Оставшиеся секторы между каталогами и staging являются журналом данных. Один сектор данных и ещё один сектор удерживаются для append/GC progress.

Примеры фактической карты:

Поле	512 КиБ	16 МиБ
Физических секторов	128	4096
Секторов таблицы в bank	1	20
Размер одного bank	4	23
Первый сектор данных	10	48
Секторов данных	100	3982
Первый сектор staging	110	4030
Секторов staging	17	65
Сектор настроек	127	4095

Таким образом, неразмеченного хвоста нет: каждый физический сектор до измеренной границы относится к locator, каталогу, данным, staging или настройкам. Области данных стираются лениво, только перед первым использованием.

Locator и смена микросхемы

Две независимые locator-копии C5FS находятся в секторах 0 и 1. Каждая хранит:

- версию формата и committed-state;
- измеренную ёмкость и format epoch;
- все рассчитанные границы областей;
- число узлов и FAT-геометрию;
- заявленную JEDEC/SFDP-границу и JEDEC ID;
- CRC32 всей записи.

На дальнем конце находится независимый C5SG guard с измеренной ёмкостью и CRC32. Он обнаруживает уменьшение flash, перенос образа на другую ёмкость и простое адресное зеркалирование.

Изменение JEDEC ID, заявленной JEDEC/SFDP-границы, измеренной ёмкости или C5 capacity guard означает замену/несоответствие микросхемы. Тогда граница измеряется заново и создаётся новая C5. Настройки при этом стираются, потому что их физический сектор мог переместиться.

Если отличается только рассчитанная новой прошивкой C5/FAT-геометрия, но совпадают chip ID, заявленная и измеренная ёмкость, дальний settings-sector и его CRC guard, деструктивный пробник не запускается: файловая часть создаётся заново, а 4080 байт настроек сохраняются. Повреждение только обоих catalog bank при валидном locator, напротив, никогда не запускает форматирование. C5 переходит в REPAIR_REQUIRED, оставляет flash неизменной и ждёт явного подтверждённого форматирования; settings-sector при этом остаётся доступен.

Каталог C5

ID узла стабилен и имеет ширину 16 бит. Таблица содержит по 20 байт на ID:

- 32-битный адрес текущей записи;
- длину данных и физическую длину записи;
- parent ID;
- first-child, next-sibling и prev-sibling;
- hash имени;

- kind/type и flags.

У файлов flags & 0x01 означает large storage, flags & 0x02 — прозрачный ZX0. data_len всегда логический размер, видимый API и FAT; record_len — физическая длина малой журнальной записи, нужная обходу и GC. Неизвестные флаги и внешний ZX0 для APP отвергаются.

Два copy-on-write bank содержат полный checkpoint. Новый checkpoint сначала стирается и заполняется в неактивном bank, защищается CRC32 и только последней операцией получает ACTIVE. Старый bank до этого остаётся валидным.

Между checkpoint находятся WAL-записи w5. В каталоге v6 запись занимает 512 байт и атомарно публикует metadata и до 16 изменённых inode; два WAL-сектора дают 16 транзакций между checkpoint. Этого достаточно для одного commit максимального APP даже на минимальном 2-КиБ FAT-кластере: основной inode, десять продолжений и изменения родительских списков. Формат v5 с записью 256 байт, девятью inode и 32 транзакциями остаётся читаемым во время миграции. Commit-state программируется последним байтом. В RAM держится не весь каталог, а 512-байтный cache таблицы и до 96 overlay-изменений. Overlay и индекс USB staging представлены отдельными плотными массивами полей: выравнивание структур не оставляет скрытых хвостов. Свободный ID ищется от запомненной подсказки, поэтому последовательное создание тысяч файлов не превращается в квадратичный просмотр начала таблицы. Когда WAL или overlay заполняется, создаётся новый checkpoint.

Каталог с inode больших файлов имеет версию 6. Физические записи файлов, staging, settings guard и основной locator сохраняют формат 5; версия каталога лежит в ранее зарезервированном байте locator. При первом запуске на томе v5 прошивка последовательно публикует два checkpoint v6 и только затем обе locator-копии. После отключения питания следующий запуск продолжает с целого банка v5 либо v6 и синхронизирует обе копии. Старые файлы, staging и настройки не перемещаются. Старая прошивка не монтирует каталог v6 и переходит в безопасный режим восстановления вместо переформатирования тома.

Поиск имени внутри одного каталога имеет сложность $O(k)$, где k -- число непосредственных детей; 16-битный hash отбрасывает несовпадающие имена до чтения строки. Последовательный просмотр каталога также $O(k)$: iterator-cache помнит следующий sibling и не начинает каждый child(i) с головы. Это намеренный выбор для жёсткой границы 4084 узла: постоянный B-tree или hash-table потребовал бы дополнительных секторов, RAM и flash-записей, не улучшая обычный просмотр папок. Host-тест почти полной 512-КиБ геометрии фиксирует линейный бюджет SPI-read и не позволяет незаметно вернуть квадратичный обход.

CRC защищает header, таблицу, WAL и каждую запись данных. При частичной записи после сбоя используется последняя полностью committed версия.

Физический журнал данных, RAW/ZX0 и отсутствие хвостов по 4 КиБ

Сектор данных имеет 16-байтный header C5D0. Далее без кластерного выравнивания упаковываются записи:

```
16-byte record header + UTF-8 basename + RAW или ZX0 payload
```

Запись не пересекает erase-сектор, но файл не получает отдельный 4-КиБ сектор. В header хранится физический stored_len; CRC покрывает header, имя и именно сохранённые байты. Для RAW stored_len == data_len, для ZX0 он меньше. Максимальный малый payload равен 1600 байтам, поэтому запись с максимальным именем занимает $16 + 31 + 1600 = 1647$ байт, поэтому даже две максимальные записи гарантированно помещаются в один сектор. Маленькие записи упаковываются плотнее. Неиспользуемым остаётся только хвост, в который уже не помещается следующая конкретная запись; фиксированной потери 4 КиБ на файл нет.

FOCAL, TinyBasic, TEXT, state, Markdown, WBMP и CHIP-8 пробуются как единый поток ZX0 с окном 256. Сжатие выбирается, только если одновременно экономит не менее 64 байт и 10%. MK61 остаётся RAW из-за горячего 64-байтного произвольного чтения; FONT остаётся RAW до перехода на raw-only FMK2; APP использует только собственное сжатие внутри контейнера.

Выбор small/large выполняется после сжатия по stored_len. Поэтому логически большой CHIP-8 может стать обычной малой записью. APP и CHIP-8 с физическим payload больше 1600 байт хранятся иначе. Обычная запись такого файла содержит небольшой дескриптор C5L0 v2: логическую и физическую длину,

поколение, CRC всего физического потока и номера секторов. Дескриптор v1 прежних RAW-файлов остаётся читаемым. Данные лежат в динамически выбранных 4-КиБ блоках C5B0; 32-байтовый header каждого блока защищает владельца, номер блока, длину, поколение и CRC payload. Это не отдельный раздел и не набор фиксированных слотов: блоки берутся из общего журнала данных и возвращаются сборщику мусора после удаления или замены большого файла.

Новая версия большого файла сначала потоково записывается в свободные C5B0. ZX0 encoder пишет туда физический поток напрямую, не создавая второй максимальный буфер. Затем дописывается дескриптор, а корневой inode и скрытые FAT-продолжения публикуются одной WAL-транзакцией. До commit старая версия остаётся единственной достижимой; после commit новые блоки становятся живыми, а старые -- мусором. Незавершённые блоки не считаются файлами и могут быть стёрты повторной записью.

Чтение ZX0 начинает декодирование с логического нуля даже для `read_range()`, потому что match зависит от предшествующей истории. Декодер хранит 256-байтное кольцевое окно, выдаёт только требуемый диапазон, но продолжает до конца и одновременно проверяет CRC физических байтов, точную логическую длину, end marker и отсутствие хвоста. Для large storage тот же reader переходит между C5B0 и проверяет CRC каждого блока. Последовательный входной поток читается временными блоками по 512 байт: блок находится на стеке только во время операции, поэтому ускорение не добавляет постоянную RAM.

Обновление и rename дописывают новую запись. Rename переносит физический payload без распаковки/повторного сжатия и пересчитывает record CRC. После WAL commit inode начинает указывать на новую запись; старая версия становится мусором. Повторная запись полностью идентичных логических данных является по-ор и не расходует ресурс flash.

GC использует отдельный reserve-sector. Поиск свободного или наиболее выгодного victim идёт окнами по 32 сектора: таблица inode читается один раз на окно, а не заново для каждого сектора. Живые записи victim копируются в reserve и их новые адреса публикуются атомарными WAL-пакетами до 16 inode. Список переносимых inode не занимает shared_scratch: GC продолжает последовательный обход каталога после каждого пакета. Поэтому он может запускаться из rename, пока там лежит сохранённый физический payload. Только после публикации всех адресов victim стирается и становится новым резервом. Потеря питания оставляет старые либо уже committed новые ссылки, но не промежуточный inode.

Host-тесты принудительно обрывают каждую отдельную program/erase-операцию на границах checkpoint, настоящего GC почти полного журнала и компактизации USB staging. После каждого возможного обрыва выполняются reboot, полная проверка старой либо новой committed версии и повтор операции. Для успешного пути также проверяются верхние пределы числа мутаций и erase, чтобы корректность не скрывала неожиданное ухудшение времени работы или износа.

Для ZX0 дополнительно выполняется fixed-seed mixed-order churn с RAW, small ZX0 и large ZX0: записи, замены, rename и delete перемешаны, каждые 13 операций моделируется reboot и всё хранилище сверяется с RAM-моделью. Замена логически многокластерного, но физически small CHIP-8 отдельно проверяется при полностью занятой квоте inode: старые FAT-extents должны переиспользоваться без временного свободного ID. Отдельная матрица перебирает обрыв после каждой flash mutation при замене small и large ZX0 и требует целиком старую либо целиком новую версию. Дополнительный граничный тест заполняет минимальный том 30 файлами и заставляет GC выбрать и стереть сектор исходной ZX0-записи непосредственно внутри rename; после reboot проверяются имя, физический размер, флаг и логические данные.

Каталоги и имена

C5 поддерживает создаваемые пользователем каталоги с произвольной структурой. Каталог хранит head двусвязного списка непосредственных детей, поэтому его просмотр не требует читать или сортировать всё оглавление flash. Максимальная глубина -- 32. Проверка учитывает всё поддерево и не позволяет обойти лимит переносом глубокого каталога.

Внутреннее имя -- валидный UTF-8 basename длиной 1--31 байт. Запрещены:

- управляющие байты и FAT-символы < > : " / \ \ | ? *;
- . и ..;
- завершающие пробел или точка;
- DOS device names CON, PRN, AUX, NUL, COM1--COM9, LPT1--LPT9, в том числе перед расширением.

В пределах каталога не допускаются имена, которые совпадают в FAT-проекции без учёта регистра. Сравнение выполняет Unicode simple case folding для ASCII, латиницы, греческого, кириллицы и ряда других регистровых диапазонов; письменности без регистра сравниваются без преобразования. Например, Каталог конфликтует с каталог, каталог demo.m61 -- с MK61-файлом basename demo, а Report.txt -- с report.txt.

Файлы получают расширение, публичную двухбайтовую сигнатуру и отдельную квоту по типу:

Тип C5	Magic	Расширение USB	Максимум	Назначение
MK61	M1	.m61	1536 байт	программа МК-61
FOCAL	F1	.foc	1536 байт	исходник FOCAL
Tiny BASIC	B2	.tbi	3584 байта	исходник Tiny BASIC
TEXT	T1	.txt	1536 байт	обычный текст
Markdown	T2	.md	1536 байт	UTF-8 Markdown
MK61 state	M2	.state.txt	1536 байт	состояние калькулятора
FONT	f1	.fmk	1536 байт F401; 8192 байта F411	графический шрифт FMK1
IMAGE1	I1	.wbmp	1600 байт	стандартный WBMP Type 0
APP	A1	.app	20 544 байта	исполняемый контейнер F401
CHIP8	C1	.ch8	3584 байта	сырой ROM CHIP-8

Magic является свойством уже известного типа C5. Он нужен для отображения типа и поиска зарегистрированного APP-обработчика, но не заменяет тип inode. Первый ASCII-байт задаёт семейство: C означает console. Вторая консольная платформа получит C2, а не ещё одно частное расширение в Проводнике. Регистр значим, поэтому F1 и f1 различаются.

IMAGE1 — только внутреннее имя типа C5. Содержимое записи не получает собственного заголовка: это неизменённый стандартный файл WBMP, который можно копировать с диска и открывать обычными программами. В short entry FAT 8.3 четырёхбуквенное .wbmp представлено как .WBM; этот вариант принимается и при обратном импорте без LFN. Формат файла, создание через ImageMagick, управление просмотрщиком и флаг его условной компиляции описаны в MK61s-mini-WBMP.pdf.

CHIP8 также не добавляет заголовок к содержимому. В .ch8 лежит стандартный ROM, который загружается по адресу 0x200; байты C1 находятся только в таблице типа и регистрации обработчика. Формат, экран, управление и критерии совместимости описаны в MK61s-mini-CHIP8.md.

APP имеет единое расширение и общий 64-байтовый заголовок MK61APP. Поле kind различает пользовательское приложение и системные сервисы FOCAL, TinyBASIC, WBMP viewer, Markdown viewer, CHIP-8 и SETUP; отдельного расширения для системных компонентов нет. Системные имена /System/FOCAL.APP, /System/BASIC.APP, /System/WBMP.APP, /System/MARKDOWN.APP, /System/CHIP8.APP и /System/SETUP.APP ищутся в обычном видимом каталоге C5, пользовательские APP можно хранить в любом другом каталоге. ABI 5 не привязан к CRC конкретной прошивки, использует таблицы с API и арендует размер своего образа/BSS, округлённый до 32 байт, сверху свободной RAM, ниже защиты стека. 20 КиБ ограничивают размер одной APP; постоянного резерва нет. После выгрузки блок возвращается в свободный участок. 0x20000000 — только виртуальная база линковки; таблица релокаций исправляет внутренние адреса при загрузке в выбранный блок. Payload сжат ZX0, при выигрыше размера применяется Thumb BCJ. После потока находится компактная таблица релокаций под общей CRC. HELP0.TXT и HELP1.TXT в /System — обычные текстовые файлы справки с подписью набора команд; их генерируют вместе с прошивкой. ABI 2/3/4 больше не исполняются. Исходный код и сборка контейнера описаны в MK61s-mini-APP-Programming.md.

Поле handled_type_magic заголовка APP может регистрировать обработчик типа. Штатный CHIP8.APP регистрирует C1. Отдельный WBMP.APP регистрирует I1, когда Markdown выключен; графический Markdown

сам обслуживает I1 и на WS0010 100x16. Графический MARKDOWN .APP регистрирует T2, а resident направляет в него также I1. Это не content magic контейнера или открываемого файла.

Проводник, редакторы и общий файловый диалог

Проводник работает с непосредственными детьми текущего каталога и не строит в RAM плоский список всего диска. Короткое ОК входит в папку, загружает M61, запускает файл соответствующего языка либо пользовательский APP. Для остальных запускаемых типов он берёт двухбайтовый magic из таблицы типа и вызывает встроенный или APP-обработчик: I1 открывает WBMP, T2 — Markdown, C1 запускает CHIP-8.

Если в папке непосредственно лежит autoexes.m61, Проводник запускает этот M61-сценарий сразу после входа в папку коротким ОК и выходит к основному экрану. Имя сравнивается без учёта регистра. Автозапуск не выполняется при открытии корня Проводника, возврате в родительскую папку через ESC и навигации внутри общих диалогов выбора файла или каталога. После выхода и повторного входа в папку сценарий запускается снова.

Долгое ОК открывает действия:

- просмотр или запуск/загрузка файла;
- редактирование FOCAL и Tiny BASIC;
- создание произвольной папки;
- переименование;
- перенос в выбранный каталог;
- удаление файла, пустой папки или подтверждённое рекурсивное удаление дерева.

Имена до 31 байта UTF-8 отображаются и прокручиваются по Unicode-символам; курсор и Cx в редакторе имени не разрезают многобайтную последовательность. Имена и строка поиска используют ту же раскладку, что языковые редакторы: цифры без префикса вводят цифры, K + цифровая клавиша включает SMS-ввод, ПП вводит пробел, F + цифровая клавиша - спецсимвол. Cx выполняет забой, а F + Cx очищает поле целиком.

FOCAL, Tiny BASIC и M61 не содержат собственных плоских браузеров. Они вызывают общий API `program_store_choose_file()`, `program_store_choose_directory()` и `program_store_choose_save_target()`. В диалоге можно переходить по дереву и создавать папки; редактор сохраняет файл в выбранный каталог. В меню Разработка -> Файлы МК-61 доступны отдельные операции открытия и сохранения текущей программы. Физические клавиши LOAD/SAVE и числовые аргументы терминальных load/save сохранены как быстрый интерфейс к корневым файлам 0.m61...99.m61.

Пути и терминал

Единый path API принимает / и \, абсолютные и относительные пути, повторные разделители, . и ... Путь с пробелами можно заключить целиком в одинарные или двойные кавычки. Расширение задаёт тип файла; имя без расширения разрешается только при единственном совпадении basename, иначе возвращается ошибка ambiguity.

Интерактивный терминал хранит текущий каталог и показывает его в prompt. Основные команды:

```
pwd
cd [path]
ls [path]                # aliases: dir, fsls
mkdir [-p] path
mv source destination
rm [-r] path             # aliases: del, fsmrm
rmdir path
df                       # alias: fsstat
save path[.m61]
load path[.m61]
open path
run path
```

Например, `open /Pictures/scheme.wbmp` вызывает тот же просмотрщик, что и короткое ОК в Проводнике. Если выключены и Markdown, и WBMP viewer, файл остаётся видимым, копируемым и переносимым, но

команда просмотра возвращает ошибку: отключение viewer не удаляет тип из C5. `open /Games/PONG.ch8` тем же маршрутом передаёт сырой ROM обработчику C1. `open /Docs/readme.md` передаёт UTF-8-документ обработчику T2: LCD1602 показывает plain text, а UC1609 или USB-экран — форматированную версию. На LCD1602 запуск WBMP и CHIP-8 возможен только при активном USB-экране; без него оболочка сообщает двумя строками: ГРАФИКА / НЕДОСТУПНА. Команда `open /Apps/demo.app` выделяет образ сверху свободной динамической RAM и вызывает его команду APPLICATION_RUN.

`mv` не перезаписывает существующий объект. `rm` удаляет файл, `rm -r --` всё дерево, `rmdir --` только пустую папку. `df` выводит измеренную ёмкость flash, занятые/свободные узлы, число файлов, каталогов и extent, виртуальный размер кластера и резерв настроек. Все изменяющие дерево команды запрещены внутри M61-сценариев явным allowlist.

Защита записи при просадке питания

На STM32F401/F411 аппаратный PVD закрывает новые записи до того, как VDD достигнет минимальных 2,7 В внешней W25Q128. При низком напряжении USB-диск динамически становится read-only: новый пакет записи или sync получает ошибку, а нижний драйвер повторно проверяет питание перед каждой командой NOR program/erase. После восстановления требуется 100 мс устойчивого питания.

Уже начатая физическая операция не прерывается и не считается гарантированно завершённой. C5 при следующем запуске принимает только последнюю полностью зафиксированную транзакцию; незавершённый batch отбрасывается штатным recovery. Состояние, события и отклонённые операции видны в строке POWER команд `df` и `prof`. Подробный контракт описан в `doc/design/STM32-power-monitor.md`.

Динамический FAT12

USB-хост видит стандартный FAT12 superfloppy без partition table:

Серийный номер FAT-тома выводится из стабильного public identity конкретного STM32 в отдельном домене. Поэтому он не меняется после reset или обновления прошивки, но различается у двух калькуляторов с одинаковой W25Q. Старый детерминированный serial по ёмкости остаётся fallback только для host-сборок и неподдерживаемых платформ без UID.

- 512 bytes per sector;
- 1 reserved sector;
- 2 FAT;
- media descriptor 0xF8;
- фиксированный root directory на 512 entries;
- volume label MK61S C5;
- строго не больше 4084 data clusters.

Порог FAT12 определяется числом кластеров, а не экспериментальной CHS-формой дискеты. Геометрии 2M и других floppy-драйверов подтверждают, что BPB может быть нестандартным, но для современных USB Mass Storage C5 сохраняет обычную семантику FAT12 и не зависит от CHS.

Размер кластера выбирается по измеренной flash:

Flash	Секторов на кластер	Кластер
128 КиБ--8 МиБ	4	2 КиБ
16 МиБ	8	4 КиБ
32 МиБ	16	8 КиБ
64 МиБ	32	16 КиБ
128 МиБ	64	32 КиБ

Это только виртуальные FAT-кластеры. Кластер 4 КиБ не означает физическую потерю 4 КиБ на файл: payload по-прежнему упакован в журнал C5.

ID узла `id` всегда показывается как `FAT cluster id + 2`. Поэтому кластеры файлов и каталогов стабильны при удалении соседних объектов. Обычный файл до 1536 байт и WBMP до 1600 байт занимает один виртуальный кластер. TinyBASIC до 3584 байт на штатном томе с 4-КиБ кластером также занимает один кластер. Большой FMK F411 использует ту же цепочку `FILE_EXTENT`, что APP и ROM. CHIP-8 ROM размером до 3584 байт и APP могут занимать несколько кластеров: первый соответствует `inode` файла, последующие - скрытым `FILE_EXTENT`. На штатной W25Q128 с кластером 4 КиБ любой `.ch8` занимает один кластер, а максимальному APP нужны один корневой и пять служебных кластеров. Это число продолжений одного файла, а не лимит количества APP или ROM. Каталог использует свой кластер; если `directory entries` не помещаются, C5 так же выделяет скрытые `extent-ID` и связывает их обычной FAT12-цепочкой.

Root directory остаётся фиксированной FAT12-областью на 512 entries (16 КиБ). Четыре заняты системно: `volume label` и LFN/short-записи маркера `.metadata_never_index`, запрещающего Spotlight индексировать диск. Для пользовательских объектов остаётся 508 entries: от 101 объекта при самом длинном имени (четыре LFN + short) до 254 объектов, чьё видимое имя занимает не более 13 UTF-16 units (одна LFN + short). C5 считает entries точно и атомарно отклоняет `create/move/rename`, который переполнил бы root; уже созданные объекты при этом не меняются. Полная квота 4084 узла доступна через произвольные подкаталоги, которые растут обычными FAT-цепочками. Компактный root принципиален для скорости: macOS может переписывать значительную часть фиксированного root при каждом создании файла.

Подкаталоги содержат стандартные `.` и `...`. Short alias строится из стабильного ID (Fxxxx для файла, Dxxxx для каталога), поэтому не коллидирует после `rename`. Пользователь всегда видит LFN.

USB staging и sync

Хост может записывать FAT, каталог и data sectors в любом порядке. Поэтому принятые 512-байтные блоки сначала объединяются в полностью ассоциативном `write-back cache`. На LCD1602/A00/A02 в нём 16 секторов -- ровно максимальный 8-КиБ пакет STM32 USB MSC. Первые 13 секторов размещены в общем 8-КиБ workspace языков, ещё три до `commit` занимают свободный `shared_scratch`. На UC1609 приостановленный внешний шрифт отдаёт USB ещё 16 секторов, итого 32.

Повторная запись одного LBA заменяет байты в том же `cache slot`; возврат к уже сохранённому содержимому снимает `dirty` без записи flash. Чтение всегда сначала проверяет `cache`, поэтому хост немедленно видит подтверждённые новые байты. Если весь USB BOT packet помещается в свободные/чистые slots, callback одним атомарным fast path копирует его в уже захваченную RAM и сразу подтверждает без SPI-чтения и перехода через `main loop`. Сверка с `persistent`-сектором откладывается до `sync` или вытеснения, поэтому повторный `zero-fill` не изнашивает NOR. Если для пакета пришлось бы вытеснить `dirty slot`, `preflight` не меняет ни одного байта и возвращает весь пакет в `deferred main-loop path`. OUT endpoint остаётся остановленным в `BUSY`, а STM32 BOT buffer передаётся туда напрямую; отдельной 8-КиБ копии пакета в `.bss` нет.

Устройство объявляет `WCE=1` стандартной SCSI Caching mode page 0x08. Как съёмный носитель (`INQUIRY.RMB=1`) оно также объявляет Removable Block Access Capabilities mode page 0x1B: один LUN (`TLUN=1`), без `system-floppy`, `FORMAT-progress` и механического changer. Ответ 0x3F возвращает обе страницы по возрастанию кода. Все `dirty slots` переносятся в `persistent staging log` при `SYNCHRONIZE CACHE`, при `START STOP UNIT` с остановкой/eject, при выходе из режима USB и при LRU вытеснении заполненного `cache`. SCSI `sync` подтверждается только после завершения переноса и атомарного C5 `commit`. Перед двухпроходным `commit` три `scratch-slot` сначала становятся `persistent`, `shared_scratch` освобождается для сборки файла, а после `commit` снова присоединяется к `cache`. Поэтому `unmount/eject OC` и `ESC` на устройстве являются точками сохранения; только аварийное питание до `sync` может потерять ещё не сброшенный `write-back`, как у обычного накопителя с `WCE`.

`START STOP UNIT` с `LOEJ=1`, `START=0` перед ответом дожидается `sync` и помечает SCSI medium извлечённым. При этом экран USB-диска не закрывается автоматически: macOS Disk Arbitration может логически извлечь том, не передав устройству ни `LOEJ`, ни иной однозначный сигнал. Тайм-аут после `SYNCHRONIZE CACHE` небезопасен, потому что та же команда используется для обычного `f sync`. Выход из режима и освобождение общих буферов всегда выполняется по `ESC`.

Persistent staging отделяет порядок FAT-транзакции хоста от атомарных операций C5. Один staging-сектор содержит 16-байтный header и семь записей вида:

```
16-byte record header + 512-byte payload
```

Header хранит LBA-key, поколение, state и CRC32. Header и 512-байтный payload программируются одной проверяемой stream-операцией; при recovery запись считается ACTIVE только после проверки CRC payload. Поэтому оборванный page-program не вытесняет предыдущее поколение и не требует отдельной commit-записи в NOR. На обычной flash до 384 последних live-блоков индексируются в RAM одним packed-массивом $LBA + record-ref$; 65-й сектор служит резервом атомарной компактизации. На 512-КиБ варианте 16 рабочих секторов дают до 96 live-блоков, а 17-й является резервом.

Перед append C5 сравнивает новый сектор с эффективным содержимым всего тома, включая data area. Повторная запись уже совпадающих FAT/root/data-блоков подтверждается без program/erase NOR. Это особенно сокращает macOS zero-fill и повторные записи AppleDouble, одновременно уменьшая износ flash.

Finder создаёт 4-КиБ AppleDouble-sidecar `._`имя для extended attributes. Такой временный файл может иметь то же расширение `.m61`, `.f0c` и т. п., но не является программой. Двухпроходный импорт распознаёт префикс `._`, не включает sidecar в C5 и не позволяет ему сорвать commit настоящего файла. Неизвестные расширения также игнорируются до проверки C5-квоты размера. После успешного sync их staged-блоки отбрасываются.

При SYNC virtual FAT выполняет два прохода:

- 1 Проверяет FAT-цепочки, LFN, уникальность cluster-ID, глубину, размеры и наличие всех data-секторов и directory tree без публикации payload. Для каждого APP этот же read-only preflight полностью проверяет заголовок, совместимость контейнера, декодирование payload и обе CRC. В ABI 5 используются ZX0, при необходимости обратный BCJ и релокации. Неполный файл или невалидный APP отклоняет текущий sync до изменения любого узла C5. Пока MSC-сеанс открыт, staging сохраняется: хост ещё может дописать, удалить или заменить отклонённый файл и повторить sync.
- 2 Создаёт/перемещает каталоги, собирает staged data и атомарно публикует файлы через C5 WAL.

Для многокластерного APP или CHIP-8 ROM второй проход не собирает файл целиком в RAM. FileSource по запросу читает нужные 512-байтовые секторы из staging, cache или прежней C5-версии и сразу передаёт их в C5B0. Поэтому порядок записи FAT не влияет на буферизацию, а частичное обновление существующей цепочки может прислать только изменившиеся секторы. Терминальный fsput использует тот же потоковый путь, сохраняя вход по 512 байт в отдельные staging-key.

Циклы FAT, выход за диапазон, повторное использование cluster-ID, неверный LFN, превышение квоты своего типа (1536 байт, 3584 для TinyBASIC, 1600 для WBMP, 1536/8192 для FMK, 3584 для CHIP-8 либо 20 544 для APP) и неподдерживаемая глубина отклоняются. При финальном выходе из USB по ESC такая детерминированно невалидная host-транзакция автоматически отменяется: staging очищается, а последний committed C5 остаётся неизменным. Ошибка возвращается и при финализации, поэтому экран не подтверждает потерянный файл как успешно сохранённый, но следующий USB-сеанс уже не блокируется старым batch. Retryable ошибки чтения, записи flash или временно недоступного ресурса staging не очищают; после reset он восстанавливается по поколениям и CRC и может быть повторно синхронизирован. Пакет из нескольких файлов не является одной общей транзакцией: после аварийного питания уже завершённые файлы могут быть новыми, остальные — старыми или отсутствовать. При этом каталог остаётся структурно валидным, а повторная передача и SYNC сходятся к полному набору. Host-тест проверяет одновременный импорт 25 APP и моделирует обрывы питания с reboot и повтором.

Настройки

Последний физический erase-сектор всегда зарезервирован под настройки:

```
offset 0..4079      settings journal
offset 4080..4095  C5SG capacity guard
```

Обычная очистка файловой системы и восстановление каталога не стирают журнал настроек. API erase-settings стирает сектор целиком и немедленно восстанавливает guard. При смене измеренной ёмкости сектор переезжает, поэтому создаётся новый пустой журнал.

Гарантии и границы

- C5 не обещает совместимость с прежними форматами тома C1--C4 и не пытается их сканировать; это не относится к сигнатуре типа CHIP-8 C1.
- Валидная C5 загружается без тестовых erase/write.

- JEDEC/SFDP -- подсказка; окончательная граница неразмеченной flash подтверждается записью и alias-aware verify.
- Полный файл и отдельная операция каталога публикуются атомарно через WAL.
- Число APP не имеет отдельного compile-time лимита; оно ограничено общей квотой inode, местом журнала данных и ёмкостью каталога FAT.
- Миграция каталога v5 в v6 сохраняет файлы, staging и настройки и продолжается после обрыва питания.
- FAT12 является представлением объектов, а не побайтовой копией SPI NOR.
- Свободные байты FAT не равны числу свободных физических байтов журнала; рабочая квота выражена числом свободных node-ID.
- CRC32 обнаруживает случайные повреждения, но не является криптографической защитой.
- При валидном locator и потере обоих catalog bank C5 не пишет во flash и не форматирует её автоматически: df предлагает явную команду format или пункт меню Стереть FLASH, а режим USB-диска не запускается.
- При физической неисправности обеих C5 locator-копий содержимое считается неразмеченным и форматируется; эвристического восстановления нет.